

Python Core Concepts - Interview Questions & Answers

1. Data Types & Structures

Q1: What's the difference between a list and a tuple?

Answer:

List: Mutable, uses square brackets [], can be modified after creation

Tuple: Immutable, uses parentheses (), cannot be modified after creation

Lists have more memory overhead due to mutability

Tuples are faster and can be used as dictionary keys (hashable)

List - Mutable

```
my_list = [1, 2, 3]
```

```
my_list[0] = 10 # Works fine
```

```
my_list.append(4) # Can add elements
```

Tuple - Immutable

```
my_tuple = (1, 2, 3)
```

```
my_tuple[0] = 10 # TypeError: 'tuple' object does not support item assignment
```

When to use:

Use lists when you need to modify the collection

Use tuples for fixed collections, function return values, or dictionary keys

Q2: Explain the difference between == and is operator.

Answer:

== checks for value equality (compares the content)

is checks for identity (compares if both variables point to the same object in memory)

Example 1: Small integers (cached by Python)

```
a = 256
```

```
b = 256
```

```
print(a == b) # True (same value)
```

```
print(a is b) # True (Python caches small integers -5 to 256)
```

Example 2: Larger integers

```
a = 1000
```

```
b = 1000
```

```
print(a == b) # True (same value)
```

```
print(a is b) # False (different objects in memory)
```

Example 3: Lists

```
list1 = [1, 2, 3]
```

```
list2 = [1, 2, 3]
```

```
list3 = list1
```

```
print(list1 == list2) # True (same content)
```

```
print(list1 is list2) # False (different objects)
```

```
print(list1 is list3) # True (same object reference)
```

Use case for 'is'

```
if x is None: # Correct way to check for None
```

```
pass
```

Q3: What are dictionary comprehensions? Give examples.

Answer: Dictionary comprehensions provide a concise way to create dictionaries.

Basic syntax: {key_expression: value_expression for item in iterable}

Example 1: Create a dictionary from a list

```
numbers = [1, 2, 3, 4, 5]
```

```
squares = {num: num**2 for num in numbers}
```

Output: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

Example 2: With condition

```
even_squares = {num: num**2 for num in numbers if num % 2 == 0}
```

Output: {2: 4, 4: 16}

Example 3: From two lists

```
keys = ['name', 'age', 'city']
```

```
values = ['John', 30, 'NYC']
```

```
person = {k: v for k, v in zip(keys, values)}
```

Output: {'name': 'John', 'age': 30, 'city': 'NYC'}

Example 4: Convert keys to uppercase

```
original = {'name': 'john', 'age': 30}
```

```
upper_keys = {k.upper(): v for k, v in original.items()}
```

Output: {'NAME': 'john', 'AGE': 30}

Example 5: Swap keys and values

```
swapped = {v: k for k, v in original.items()}
```

Output: {'john': 'name', 30: 'age'}

Q4: Explain mutable vs immutable objects with examples.

Answer:

Immutable objects: Cannot be changed after creation

int, float, bool, str, tuple, frozenset

Mutable objects: Can be changed after creation

list, dict, set, user-defined classes (by default)

Immutable - String

```
s = "hello"
```

```
print(id(s)) # Memory address: 140234567890
```

```
s = s + " world" # Creates a NEW object  
print(id(s)) # Different memory address: 140234567900
```

Immutable - Tuple

```
t = (1, 2, 3)
```

t[0] = 10 # TypeError: 'tuple' object does not support item assignment

Mutable - List

```
lst = [1, 2, 3]  
print(id(lst)) # Memory address: 140234560000  
lst.append(4) # Modifies the SAME object  
print(id(lst)) # Same memory address: 140234560000
```

Gotcha with mutable default arguments

```
def add_item(item, items=[]): # DANGER: Mutable default
```

```
    items.append(item)  
  
    return items
```

```
print(add_item(1)) # [1]  
print(add_item(2)) # [1, 2] - Not what you expected!  
print(add_item(3)) # [1, 2, 3] - The list persists!
```

Correct way

```
def add_item_correct(item, items=None):
```

```
    if items is None:

        items = []

    items.append(item)

    return items
```

Why this matters:

Immutable objects are thread-safe

Immutable objects can be dictionary keys

Understanding this prevents bugs with default arguments and copying

Q5: What is the difference between shallow copy and deep copy?

Answer:

Shallow Copy: Creates a new object but references to nested objects are shared

Deep Copy: Creates a new object and recursively copies all nested objects

```
import copy
```

Original list with nested list

```
original = [1, 2, [3, 4], 5]
```

Shallow copy

```
shallow = copy.copy(original)
```

or

```
shallow = original.copy()
```

or

```
shallow = original[:]
```

Deep copy

```
deep = copy.deepcopy(original)
```

Modify nested list

```
original[2][0] = 999
```

```
print("Original:", original) # [1, 2, [999, 4], 5]
```

```
print("Shallow:", shallow) # [1, 2, [999, 4], 5] - nested list is shared!
```

```
print("Deep:", deep) # [1, 2, [3, 4], 5] - completely independent
```

Example with dictionaries

```
original_dict = {'a': 1, 'b': {'c': 2}}
```

```
shallow_dict = original_dict.copy()
```

```
deep_dict = copy.deepcopy(original_dict)
```

```
original_dict['b']['c'] = 999
```

```
print("Original:", original_dict) # {'a': 1, 'b': {'c': 999}}
```

```
print("Shallow:", shallow_dict) # {'a': 1, 'b': {'c': 999}} - shared nested dict
```

```
print("Deep:", deep_dict) # {'a': 1, 'b': {'c': 2}} - independent
```

When to use:

Use shallow copy when objects don't contain nested mutable objects

Use deep copy when you need complete independence from the original

Q6: Explain the collections module and its useful data structures.

Answer:

```
from collections import defaultdict, Counter, deque, namedtuple, OrderedDict
```

1. defaultdict - Never raises KeyError

```
word_count = defaultdict(int)
```

```
for word in ['apple', 'banana', 'apple']:
```

```
    word_count[word] += 1 # No need to check if key exists
```

```
print(word_count) # defaultdict(<class 'int'>, {'apple': 2, 'banana': 1})
```

With list

```
groups = defaultdict(list)
```

```
for name, group in [('John', 'A'), ('Jane', 'B'), ('Bob', 'A')]:
```

```
    groups[group].append(name)
```

```
print(groups) # defaultdict(<class 'list'>, {'A': ['John', 'Bob'], 'B': ['Jane']})
```

2. Counter - Count hashable objects

```
from collections import Counter
```

```
words = ['apple', 'banana', 'apple', 'cherry', 'banana', 'apple']
```

```
counter = Counter(words)
```

```
print(counter) # Counter({'apple': 3, 'banana': 2, 'cherry': 1})
```

```
print(counter.most_common(2)) # [('apple', 3), ('banana', 2)]
```

Counter operations

```
c1 = Counter(['a', 'b', 'c', 'a'])
```



```
c2 = Counter(['a', 'b', 'd'])  
  
print(c1 + c2) # Counter({'a': 3, 'b': 2, 'c': 1, 'd': 1})  
  
print(c1 - c2) # Counter({'a': 1, 'c': 1})
```

3. deque - Double-ended queue (efficient for both ends)

```
dq = deque([1, 2, 3])  
  
dq.append(4) # Add to right: deque([1, 2, 3, 4])  
  
dq.appendleft(0) # Add to left: deque([0, 1, 2, 3, 4])  
  
dq.pop() # Remove from right: 4  
  
dq.popleft() # Remove from left: 0
```

deque([1, 2, 3])

Rotating

```
dq.rotate(1) # deque([3, 1, 2])  
  
dq.rotate(-1) # deque([1, 2, 3])
```

4. namedtuple - Tuple with named fields

```
Point = namedtuple('Point', ['x', 'y'])  
  
p = Point(10, 20)  
  
print(p.x, p.y) # 10 20  
  
print(p[0], p[1]) # 10 20 (still accessible by index)
```

More readable than regular tuples

```
Person = namedtuple('Person', ['name', 'age', 'city'])
```

```
john = Person('John', 30, 'NYC')
```

```
print(john.name) # Much better than john[0]
```

5. OrderedDict - Maintains insertion order (Python 3.7+ dicts also maintain order)

Still useful for: `move_to_end()`, `popitem(last=False)`

```
od = OrderedDict()
```

```
od['a'] = 1
```

```
od['b'] = 2
```

```
od['c'] = 3
```

```
od.move_to_end('a') # Move 'a' to the end
```

```
print(od) # OrderedDict([('b', 2), ('c', 3), ('a', 1)])
```

Use Cases:

defaultdict: Grouping, counting without checking key existence

Counter: Frequency counting, finding most common elements

deque: Queue/stack operations, sliding window problems

namedtuple: Readable tuple alternatives, lightweight classes

OrderedDict: When order manipulation matters (`move_to_end`)

2. Functions

Q7: Explain `*args` and `**kwargs` with examples.

Answer:

*args allows a function to accept any number of positional arguments **kwargs allows a function to accept any number of keyword arguments

*args - Variable positional arguments

```
def sum_all(*args):
```

```
    """Sum any number of arguments"""

    print(f'args type: {type(args)}') # <class 'tuple'>

    return sum(args)
```

```
print(sum_all(1, 2, 3)) # 6
```

```
print(sum_all(1, 2, 3, 4, 5)) # 15
```

**kwargs - Variable keyword arguments

```
def print_info(**kwargs):
```

```
    """Print any number of keyword arguments"""

    print(f'kwargs type: {type(kwargs)}') # <class 'dict'>

    for key, value in kwargs.items():

        print(f'{key}: {value}')
```

```
print_info(name="John", age=30, city="NYC")
```

name: John

age: 30

city: NYC

Combining regular args, *args, and **kwargs

```
def full_example(required, *args, default="default", **kwargs):
```

```
    print(f"Required: {required}")
```

```
    print(f"Args: {args}")
```

```
    print(f"Default: {default}")
```

```
    print(f"Kwargs: {kwargs}")
```

```
full_example("must_have", 1, 2, 3, default="custom", extra="info", more="data")
```

Required: must_have

Args: (1, 2, 3)

Default: custom

Kwargs: {'extra': 'info', 'more': 'data'}

Unpacking arguments

```
def add(a, b, c):
```

```
return a + b + c
```

```
numbers = [1, 2, 3]
```

```
print(add(*numbers)) # Unpacking list: 6
```

```
kwargs_dict = {'a': 1, 'b': 2, 'c': 3}
```

```
print(add(**kwargs_dict)) # Unpacking dict: 6
```

Real-world example: Wrapper function

```
def logged_function(func):
```

```
    def wrapper(*args, **kwargs):

        print(f"Calling {func.__name__} with args={args}, kwargs={kwargs}")

        result = func(*args, **kwargs)

        print(f"Result: {result}")

        return result

    return wrapper
```

```
@logged_function
```

```
def multiply(x, y):
```

```
    return x * y
```

```
multiply(3, 4)
```

Calling multiply with args=(3, 4), kwargs={}

Result: 12

Q8: What are decorators? Explain with examples.

Answer:

A decorator is a function that takes another function and extends its behavior without explicitly modifying it.

Basic decorator

```
def my_decorator(func):
```

```
    def wrapper():

        print("Before function call")

        func()

        print("After function call")

    return wrapper
```

```
@my_decorator
```

```
def say_hello():
```

```
    print("Hello!")
```

```
say_hello()
```

Output:

Before function call

Hello!

After function call

Decorator with arguments

```
def repeat(times):
```

```
    def decorator(func):

        def wrapper(*args, **kwargs):

            for _ in range(times):

                result = func(*args, **kwargs)

            return result

        return wrapper

    return decorator
```

```
@repeat(times=3)
```

```
def greet(name):
```

```
    print(f"Hello {name}")
```

```
greet("John")
```

Output:

Hello John

Hello John

Hello John

Practical example: Timer decorator

```
import time
```

```
from functools import wraps
```

```
def timer(func):
```

```
    @wraps(func) # Preserves original function metadata
```

```
    def wrapper(*args, **kwargs):
```

```
        start = time.time()
```

```
        result = func(*args, **kwargs)
```

```
        end = time.time()
```

```
        print(f'{func.__name__} took {end - start:.4f} seconds')
```

```
        return result
```

```
    return wrapper
```

```
@timer
```

```
def slow_function():
```



```
time.sleep(1)

return "Done"
```

```
slow_function()
```

slow_function took 1.0001 seconds

Authentication decorator example

```
def require_auth(func):
```

```
    @wraps(func)

    def wrapper(user, *args, **kwargs):

        if not user.get('is_authenticated'):

            raise PermissionError("User not authenticated")

        return func(user, *args, **kwargs)

    return wrapper
```

```
@require_auth
```

```
def view_profile(user):
```

```
    return f"Profile of {user['name']}"
```

Multiple decorators (applied bottom to top)

def bold(func):

```
def wrapper():  
  
    return f"<b>{func()}</b>"  
  
return wrapper
```

def italic(func):

```
def wrapper():  
  
    return f"<i>{func()}</i>"  
  
return wrapper
```

@bold

@italic

def greet():

```
return "Hello"
```

print(greet()) # *Hello*

Class-based decorator

class CountCalls:

```
def __init__(self, func):  
  
    self.func = func  
  
    self.count = 0  
  
  
def __call__(self, *args, **kwargs):
```

```
self.count += 1

print(f'Call {self.count} of {self.func.__name__}')

return self.func(*args, **kwargs)
```

@CountCalls

def say_hi():

```
print("Hi!")
```

say_hi() # Call 1 of say_hi

say_hi() # Call 2 of say_hi

Common Use Cases:

Logging

Authentication/Authorization

Caching/Memoization

Rate limiting

Input validation

Performance monitoring

Q9: Explain closures with examples.

Answer:

A closure is a function object that remembers values in enclosing scopes even if they are not present in memory.

Basic closure

def outer_function(x):

```
# x is in the enclosing scope
```

```
def inner_function(y):
```

```
# inner_function "closes over" x
```

```
    return x + y

    return inner_function
```

```
add_5 = outer_function(5)

add_10 = outer_function(10)

print(add_5(3)) # 8 (5 + 3)

print(add_10(3)) # 13 (10 + 3)
```

Closure maintains state

```
def counter():
```

```
    count = 0

    def increment():

        nonlocal count # Access the enclosing scope variable

        count += 1

        return count

    return increment
```

```
c1 = counter()

c2 = counter()

print(c1()) # 1

print(c1()) # 2

print(c1()) # 3

print(c2()) # 1 (separate instance)
```

Practical example: Function factory

```
def make_multiplier(n):
```

```
    def multiplier(x):  
  
        return x * n  
  
    return multiplier
```

```
times_3 = make_multiplier(3)
```

```
times_5 = make_multiplier(5)
```

```
print(times_3(10)) # 30
```

```
print(times_5(10)) # 50
```

Closure in decorators

```
def logger(prefix):
```

```
    def decorator(func):  
  
        def wrapper(*args, **kwargs):  
  
            print(f'{prefix}: Calling {func.__name__}')  
  
            return func(*args, **kwargs)  
  
        return wrapper  
  
    return decorator
```

```
@logger("[INFO]")
```

```
def add(a, b):
```

```
    return a + b
```

```
add(2, 3) # [INFO]: Calling add
```

Data hiding/encapsulation

```
def create_account(initial_balance):
```

```
    balance = initial_balance
```

```
    def deposit(amount):
```

```
        nonlocal balance
```

```
        balance += amount
```

```
        return balance
```

```
    def withdraw(amount):
```

```
        nonlocal balance
```

```
        if amount > balance:
```

```
            return "Insufficient funds"
```

```
        balance -= amount
```

```
        return balance
```

```
    def get_balance():
```

```
        return balance
```

```
return deposit, withdraw, get_balance
```

```
deposit, withdraw, get_balance = create_account(100)
```

```
print(deposit(50)) # 150
```

```
print(withdraw(30)) # 120
```

```
print(get_balance()) # 120
```

balance is not directly accessible - encapsulated!

Key Points:

Closures remember the environment in which they were created

They provide data hiding and encapsulation

Commonly used in decorators and callbacks

Use nonlocal keyword to modify enclosing scope variables

Q10: What is the difference between map(), filter(), and reduce()?

Answer:

```
from functools import reduce
```

map() - Apply a function to all items

```
numbers = [1, 2, 3, 4, 5]
```

Traditional approach

```
squares = []
```

```
for n in numbers:
```

```
squares.append(n ** 2)
```

Using map

```
squares = list(map(lambda x: x ** 2, numbers))  
  
print(squares) # [1, 4, 9, 16, 25]
```

map with multiple iterables

```
list1 = [1, 2, 3]  
  
list2 = [10, 20, 30]  
  
result = list(map(lambda x, y: x + y, list1, list2))  
  
print(result) # [11, 22, 33]
```

filter() - Filter items based on a condition

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Traditional approach

```
evens = []
```

```
for n in numbers:
```

```
    if n % 2 == 0:  
  
        evens.append(n)
```

Using filter

```
evens = list(filter(lambda x: x % 2 == 0, numbers))  
  
print(evens) # [2, 4, 6, 8, 10]
```


Filter out None/empty values

```
data = ['', 'hello', None, 'world', 0, False, 'python']
```

```
cleaned = list(filter(None, data))
```

```
print(cleaned) # ['hello', 'world', 'python']
```

reduce() - Reduce a list to a single value

```
numbers = [1, 2, 3, 4, 5]
```

Traditional approach

```
total = 0
```

```
for n in numbers:
```

```
    total += n
```

Using reduce

```
total = reduce(lambda x, y: x + y, numbers)
```

```
print(total) # 15
```

Find maximum

```
numbers = [3, 7, 2, 9, 1]
```

```
maximum = reduce(lambda x, y: x if x > y else y, numbers)
```

```
print(maximum) # 9
```

Combining map, filter, reduce

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Sum of squares of even numbers

```
result = reduce(
```

```
    lambda x, y: x + y,  
  
    map(lambda x: x ** 2,  
  
        filter(lambda x: x % 2 == 0, numbers))
```

```
)
```

```
print(result) # 220 (4 + 16 + 36 + 64 + 100)
```

Modern alternative: List comprehension (often more readable)

```
result = sum(x ** 2 for x in numbers if x % 2 == 0)
```

```
print(result) # 220
```

Comparison:

Function

Purpose

Returns

Example

map()

Transform each item

Iterator of transformed items

map(lambda x: x*2, [1,2,3]) [2,4,6]

filter()

Keep items matching condition

Iterator of filtered items

```
filter(lambda x: x>2, [1,2,3,4])  [3,4]
```

reduce()

Combine items to single value

Single value

```
reduce(lambda x,y: x+y, [1,2,3])  6
```

When to use:

List comprehensions are often more Pythonic and readable

Use map/filter/reduce when working with existing functions

Useful in functional programming patterns

3. Object-Oriented Programming

Q11: Explain the four pillars of OOP with Python examples.

Answer:

4. Encapsulation - Bundling data and methods, hiding internal details

class BankAccount:

```
def __init__(self, balance):

    self.__balance = balance # Private attribute (name mangling)


def deposit(self, amount):

    if amount > 0:

        self.__balance += amount

        return True

    return False


def get_balance(self):

    return self.__balance
```

```
# Cannot access __balance directly from outside
```

```
# But can access via _BankAccount__balance (name mangling)
```

```
account = BankAccount(1000)
```

```
account.deposit(500)
```

```
print(account.get_balance()) # 1500
```

print(account.__balance) # AttributeError

2. Abstraction - Hiding complex implementation, showing only necessary features

```
from abc import ABC, abstractmethod
```

```
class Shape(ABC): # Abstract base class
```

```
@abstractmethod
```

```
def area(self):
```

```
    pass
```

```
@abstractmethod
```

```
def perimeter(self):
```

```
    pass
```

```
class Rectangle(Shape):
```

```
def __init__(self, width, height):
```

```
    self.width = width
```

```
self.height = height
```

```
def area(self):
```

```
    return self.width * self.height
```

```
def perimeter(self):
```

```
    return 2 * (self.width + self.height)
```

```
class Circle(Shape):
```

```
    def __init__(self, radius):
```

```
        self.radius = radius
```

```
    def area(self):
```

```
        return 3.14159 * self.radius ** 2
```

```
    def perimeter(self):
```

```
        return 2 * 3.14159 * self.radius
```

**shape = Shape() # TypeError: Can't
instantiate abstract class**

```
rect = Rectangle(5, 3)
```

```
circle = Circle(4)
```

```
shapes = [rect, circle]
```

```
for shape in shapes:
```

```
    print(f"Area: {shape.area()}") # Polymorphism in action
```

3. Inheritance - Creating new classes from existing ones

```
class Animal:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    def speak(self):
```

```
        pass
```

```
class Dog(Animal):
```

```
    def speak(self):
```

```
        return f"{self.name} says Woof!"
```

```
class Cat(Animal):
```

```
    def speak(self):
```

```
        return f"{self.name} says Meow!"
```

Multiple Inheritance

```
class Flyable:
```

```
def fly(self):  
  
    return "Flying..."
```

```
class Swimmable:
```

```
    def swim(self):  
  
        return "Swimming..."
```

```
class Duck(Animal, Flyable, Swimmable):
```

```
    def speak(self):  
  
        return f"{self.name} says Quack!"
```

```
dog = Dog("Buddy")  
  
cat = Cat("Whiskers")  
  
duck = Duck("Donald")  
  
print(dog.speak()) # Buddy says Woof!  
  
print(duck.speak()) # Donald says Quack!  
  
print(duck.fly()) # Flying...  
  
print(duck.swim()) # Swimming...
```

4. Polymorphism - Same interface, different implementations

Method Overriding

```
class Payment:
```

```
    def process(self):  
  
        pass
```

```
class CreditCard(Payment):
```

```
    def process(self):  
  
        return "Processing credit card payment"
```

```
class PayPal(Payment):
```

```
    def process(self):  
  
        return "Processing PayPal payment"
```

```
class Bitcoin(Payment):
```

```
    def process(self):  
  
        return "Processing Bitcoin payment"
```

Polymorphism: Same method name, different behavior

```
payments = [CreditCard(), PayPal(), Bitcoin()]
```

```
for payment in payments:
```

```
    print(payment.process())
```

Operator Overloading (another form of polymorphism)

```
class Vector:
```

```
    def __init__(self, x, y):
```



```
self.x = x

self.y = y

def __add__(self, other):

    return Vector(self.x + other.x, self.y + other.y)

def __str__(self):

    return f"Vector({self.x}, {self.y})"
```

```
v1 = Vector(2, 3)
```

```
v2 = Vector(4, 5)
```

```
v3 = v1 + v2 # Uses add
```

```
print(v3) # Vector(6, 8)
```

Q12: Explain Method Resolution Order (MRO) and the Diamond Problem.

Answer:

MRO determines the order in which base classes are searched when looking for a method. Python uses C3 Linearization algorithm.

Simple inheritance

```
class A:
```

```
    def method(self):

        return "A"
```

```
class B(A):
```

```
def method(self):  
  
    return "B"
```

```
class C(A):
```

```
def method(self):  
  
    return "C"
```

```
class D(B, C):
```

```
    pass
```

```
d = D()
```

```
print(d.method()) # "B" - B comes before C in MRO
```

```
print(D.mro)
```

**(<class 'main.D'>, <class 'main.B'>, <class 'main.C'>,
<class 'main.A'>, <class 'object'>)**

<class 'main.A'>, <class 'object'>)

Diamond Problem

```
class A:
```

```
def method(self):  
  
    print("A")
```

```
class B(A):
```

```
def method(self):  
  
    print("B")  
  
    super().method()
```

```
class C(A):
```

```
def method(self):  
  
    print("C")  
  
    super().method()
```

```
class D(B, C):
```

```
def method(self):  
  
    print("D")  
  
    super().method()
```

```
d = D()
```

```
d.method()
```

Output:

D

B

C

A

MRO: D -> B -> C -> A -> object

```
print(D.mro())
```

Real-world example

```
class LoggingMixin:
```

```
    def log(self, message):  
  
        print(f"LOG: {message}")
```

```
class TimestampMixin:
```

```
    def get_timestamp(self):  
  
        from datetime import datetime  
  
        return datetime.now()
```

```
class User(LoggingMixin, TimestampMixin):
```

```
    def __init__(self, name):  
  
        self.name = name  
  
        self.log(f"User {name} created at {self.get_timestamp()}")
```

```
user = User("John")
```

**LOG: User John created at 2024-04-18
10:30:45.123456**

Check MRO

```
print(User.mro())
```

**[<class 'User'>, <class 'LoggingMixin'>,
<class 'TimestampMixin'>, <class
'object'>]**

Key Points:

MRO follows depth-first, left-to-right pattern

Use `super()` for cooperative multiple inheritance

Check MRO with `ClassName.mro()` or `ClassName.mro`

C3 linearization ensures consistency

Q13: What are class methods, static methods, and instance methods?

Answer:

class MyClass:

```
class_variable = "I'm a class variable"
```

```
def __init__(self, value):
```

```
    self.instance_variable = value
```

```
    # Instance method - has access to instance (self)
```

```
def instance_method(self):

    return f"Instance method called, value: {self.instance_variable}"


# Class method - has access to class (cls), not instance

@classmethod

def class_method(cls):

    return f"Class method called, class var: {cls.class_variable}"


# Static method - no access to instance or class

@staticmethod

def static_method(x, y):

    return f"Static method called with {x} and {y}"


# Class method as alternative constructor

@classmethod

def from_string(cls, string_value):

    value = int(string_value)

    return cls(value)
```

Usage

```
obj = MyClass(42)
```

Instance method - needs an instance

```
print(obj.instance_method()) # Instance method called, value: 42
```

Class method - can be called on class or instance

```
print(MyClass.class_method()) # Class method called, class var: I'm a class variable
```

```
print(obj.class_method()) # Same output
```

Static method - can be called on class or instance

```
print(MyClass.static_method(1, 2)) # Static method called with 1 and 2
```

```
print(obj.static_method(1, 2)) # Same output
```

Alternative constructor using class method

```
obj2 = MyClass.from_string("100")
```

```
print(obj2.instance_variable) # 100
```

Real-world example: Date class

```
from datetime import datetime
```

```
class Date:
```

```
def __init__(self, year, month, day):
```

```
    self.year = year
```

```
    self.month = month
```

```
    self.day = day
```

```
@classmethod
```

```
def from_string(cls, date_string):
```

```
    """Alternative constructor from string format"""
```

```
    year, month, day = map(int, date_string.split('-'))
```

```
    return cls(year, month, day)
```

```
@classmethod
```

```
def today(cls):
```

```
    """Alternative constructor for today's date"""
```

```
    now = datetime.now()
```

```
    return cls(now.year, now.month, now.day)
```

```
@staticmethod
```

```
def is_valid_date(year, month, day):
```

```
    """Utility function - doesn't need instance or class"""
```



```
return 1 <= month <= 12 and 1 <= day <= 31
```

```
def __str__(self):
```

```
    return f"{self.year}-{self.month:02d}-{self.day:02d}"
```

Different ways to create Date objects

```
date1 = Date(2024, 4, 18)
```

```
date2 = Date.from_string("2024-04-18")
```

```
date3 = Date.today()
```

```
print(date1) # 2024-04-18
```

```
print(date2) # 2024-04-18
```

```
print(date3) # 2024-04-18
```

Static method usage

```
print(Date.is_valid_date(2024, 13, 1)) # False
```

When to use:

Instance method: When you need to access/modify instance data

Class method: Alternative constructors, factory methods, accessing class variables

Static method: Utility functions related to the class but don't need instance/class data

Q14: Explain magic/dunder methods with common examples.

Answer:

Magic methods (double underscore methods) allow you to define behavior for built-in operations.

class Book:

```
def __init__(self, title, author, pages):

    """Constructor - called when object is created"""

    self.title = title

    self.author = author

    self.pages = pages


def __str__(self):

    """String representation for users - used by str() and print()"""

    return f"{self.title} by {self.author}"


def __repr__(self):

    """Official representation for developers - used by repr()"""

    return f"Book('{self.title}', '{self.author}', {self.pages})"


def __len__(self):

    """Length - used by len()"""

    return self.pages


def __eq__(self, other):

    """Equality - used by =="""
```

```
return self.title == other.title and self.author == other.author
```

```
def __lt__(self, other):
```

```
    """Less than - used by <"""
```

```
    return self.pages < other.pages
```

```
def __add__(self, other):
```

```
    """Addition - used by +"""
```

```
    return self.pages + other.pages
```

```
def __getitem__(self, key):
```

```
    """Indexing - used by book[key]"""
```

```
    if key == 'title':
```

```
        return self.title
```

```
    elif key == 'author':
```

```
        return self.author
```

```
    elif key == 'pages':
```

```
        return self.pages
```

```
    raise KeyError(key)
```

```
def __call__(self):  
  
    """Makes object callable - used by book()"""  
  
    return f"Reading {self.title}"
```

Usage

```
book1 = Book("Python Tricks", "Dan Bader", 300)  
  
book2 = Book("Fluent Python", "Luciano Ramalho", 792)  
  
print(book1) # str: Python Tricks by Dan Bader  
  
print(repr(book1)) # repr: Book('Python Tricks', 'Dan Bader', 300)  
  
print(len(book1)) # len: 300  
  
print(book1 == book2) # eq: False  
  
print(book1 < book2) # lt: True  
  
print(book1 + book2) # add: 1092  
  
print(book1['title']) # getitem: Python Tricks  
  
print(book1()) # call: Reading Python Tricks
```

Container magic methods

```
class CustomList:
```

```
    def __init__(self):  
  
        self.items = []  
  
  
    def __len__(self):  
  
        return len(self.items)
```

```
def __getitem__(self, index):
```

```
    return self.items[index]
```

```
def __setitem__(self, index, value):
```

```
    self.items[index] = value
```

```
def __delitem__(self, index):
```

```
    del self.items[index]
```

```
def __contains__(self, item):
```

```
    """Membership test - used by 'in'"""
```

```
    return item in self.items
```

```
def __iter__(self):
```

```
    """Make object iterable"""
```

```
    return iter(self.items)
```

```
def append(self, item):
```

```
    self.items.append(item)
```

```
my_list = CustomList()

my_list.append(1)

my_list.append(2)

my_list.append(3)

print(len(my_list)) # 3

print(my_list[0]) # 1

my_list[0] = 10 # setitem

print(2 in my_list) # True (contains)

for item in my_list: # iter
```

```
    print(item)
```

Context manager magic methods

```
class FileManager:
```

```
    def __init__(self, filename, mode):

        self.filename = filename

        self.mode = mode

        self.file = None

    def __enter__(self):

        """Called when entering 'with' block"""

        self.file = open(self.filename, self.mode)

        return self.file
```

```
def __exit__(self, exc_type, exc_val, exc_tb):

    """Called when exiting 'with' block"""

    if self.file:

        self.file.close()

    # Return False to propagate exceptions, True to suppress

    return False
```

Usage

with FileManager('test.txt', 'w') as f:

```
f.write('Hello World')
```

File automatically closed after 'with' block

Most Common Magic Methods:

Method

Operation

Example

init

Constructor

obj = MyClass()

str

String representation

str(obj), print(obj)

repr

Official representation

`repr(obj)`

len

Length

`len(obj)`

getitem

Indexing

`obj[key]`

setitem

Assignment

`obj[key] = value`

eq

Equality

`obj1 == obj2`

lt

Less than

`obj1 < obj2`

add

Addition

`obj1 + obj2`

call

Call object

`obj()`

enter/exit

Context manager

with obj:

iter/next

Iterator

for x in obj:

Q15: What is the difference between **str** and **repr**?

Answer:

```
import datetime
```

```
class Person:
```

```
    def __init__(self, name, age):
```

```
        self.name = name
```



```
self.age = age
```

```
def __str__(self):
```

```
    # User-friendly, readable representation
```

```
    # Used by print() and str()
```

```
    return f"{self.name}, {self.age} years old"
```

```
def __repr__(self):
```

```
    # Developer-friendly, unambiguous representation
```

```
    # Should be valid Python code to recreate the object
```

```
    # Used by repr() and in interactive interpreter
```

```
    return f"Person(name='{self.name}', age={self.age})"
```

```
person = Person("John", 30)
```

str - for end users

```
print(str(person)) # John, 30 years old
```

```
print(person) # John, 30 years old (print calls str)
```

repr - for developers

```
print(repr(person)) # Person(name='John', age=30)
```

```
print([person]) # [Person(name='John', age=30)] (containers use repr)
```

Interactive interpreter uses repr

```
>>> person
```

```
Person(name='John', age=30)
```

Built-in example: datetime

```
now = datetime.datetime.now()
```

```
print(str(now)) # 2024-04-18 10:30:45.123456 (readable)
```

```
print(repr(now)) # datetime.datetime(2024, 4, 18, 10, 30, 45, 123456) (recreatable)
```

If str is not defined, Python falls back to repr

```
class Product:
```

```
    def __init__(self, name, price):
```

```
        self.name = name
```

```
        self.price = price
```

```
    def __repr__(self):
```

```
        return f"Product('{self.name}', ${self.price})"
```

```
product = Product("Laptop", 999)
```

```
print(product) # Product('Laptop', $999)

print(str(product)) # Product('Laptop', $999) (fallback to repr)

print(repr(product)) # Product('Laptop', $999)
```

Best practice

class BestPractice:

```
def __init__(self, value):

    self.value = value


def __repr__(self):

    # Always define __repr__ to return valid Python

    return f"{self.__class__.__name__}({self.value!r})"


def __str__(self):

    # Define __str__ for user-friendly output

    return f"Value: {self.value}"
```

Key Differences:

Aspect

str

repr

Purpose

User-friendly

Developer-friendly

Audience

End users

Developers

Goal

Readable

Unambiguous

Used by

`str()`, `print()`

`repr()`, containers, interactive shell

Should return

Readable string

Valid Python code (ideally)

Fallback

Falls back to **repr**

No fallback

Rule of Thumb:

Always implement **repr**

Implement **str** only if you need a different user-facing representation

repr's output should let you recreate the object: `eval(repr(obj)) == obj`

4. Iterators & Generators

Q16: What is the difference between an iterator and an iterable?

Answer:

Iterable: An object that can return an iterator (has **iter** method) Iterator: An object that produces values one at a time (has **iter** and **next** methods)

List is an iterable, not an iterator

```
my_list = [1, 2, 3]
```

```
print(hasattr(my_list, 'iter')) # True - iterable
```

```
print(hasattr(my_list, 'next')) # False - not an iterator
```

Get iterator from iterable

```
my_iterator = iter(my_list) # or my_list.iter()
```

```
print(hasattr(my_iterator, 'iter')) # True
```

```
print(hasattr(my_iterator, 'next')) # True - iterator
```

Using iterator

```
print(next(my_iterator)) # 1
```

```
print(next(my_iterator)) # 2
```

```
print(next(my_iterator)) # 3
```

`print(next(my_iterator))` # StopIteration exception

Custom iterable

```
class Countdown:
```

```
    """An iterable that counts down from start to 0"""
```

```
    def __init__(self, start):
```

```
        self.start = start
```

```
    def __iter__(self):
```

```
        """Return an iterator"""
```

```
        return CountdownIterator(self.start)
```

```
class CountdownIterator:
```

```
    """Iterator for Countdown"""
```

```
def __init__(self, start):

    self.current = start


def __iter__(self):

    """Iterator must return itself"""

    return self


def __next__(self):

    """Return next value or raise StopIteration"""

    if self.current < 0:

        raise StopIteration

    value = self.current

    self.current -= 1

    return value
```

Usage

```
countdown = Countdown(5)
```

```
for num in countdown:
```

```
    print(num) # 5, 4, 3, 2, 1, 0
```

Can create multiple independent iterators

```
iter1 = iter(countdown)

iter2 = iter(countdown)

print(next(iter1)) # 5

print(next(iter2)) # 5 (independent)
```

Simpler: Combine iterable and iterator in one class

class CountdownSimple:

```
    def __init__(self, start):

        self.start = start

        self.current = start

    def __iter__(self):

        self.current = self.start # Reset

        return self

    def __next__(self):

        if self.current < 0:

            raise StopIteration
```

```
value = self.current
```

```
self.current -= 1
```

```
return value
```

```
countdown = CountdownSimple(3)
```

```
for num in countdown:
```

```
    print(num) # 3, 2, 1, 0
```

Iterable vs Iterator: Key difference

```
numbers = [1, 2, 3]
```

Can iterate multiple times over an iterable

```
for n in numbers:
```

```
    print(n)
```

```
for n in numbers: # Works fine
```

```
    print(n)
```

But iterator gets exhausted

```
numbers_iter = iter(numbers)
```

```
for n in numbers_iter:
```



```
print(n)
```

for n in numbers_iter: # Nothing printed - iterator exhausted

```
print(n)
```

Summary:

Iterable: Container that provides an iterator via **iter()** (lists, tuples, dicts, sets, strings)

Iterator: Object with state that provides values via **next()** and raises StopIteration when done

Every iterator is an iterable, but not every iterable is an iterator

Iterators are exhaustible, iterables are not

Q17: Explain generators and the yield keyword.

Answer:

Generators are functions that return an iterator using yield. They generate values lazily (on-demand) rather than storing all values in memory.

Regular function - returns all values at once

```
def get_squares_list(n):
```

```
    result = []
```

```
    for i in range(n):
```

```
        result.append(i ** 2)
```

```
    return result
```

Generator function - yields values one at a time

```
def get_squares_generator(n):
```

```
    for i in range(n):
```

```
        yield i ** 2
```

Usage

```
squares_list = get_squares_list(5)
```

```
print(squares_list) # [0, 1, 4, 9, 16] - all in memory
```

```
squares_gen = get_squares_generator(5)
```

```
print(squares_gen) # - nothing computed yet
```

Iterate through generator

```
for square in squares_gen:
```

```
    print(square) # Computed on-the-fly
```

Generator vs List: Memory efficiency

```
import sys
```

```
list_comp = [x ** 2 for x in range(1000000)]
```

```
gen_exp = (x ** 2 for x in range(1000000))
```

```
print(sys.getsizeof(list_comp)) # ~8000000 bytes
```

```
print(sys.getsizeof(gen_exp)) # ~120 bytes
```

Generator with multiple yields

```
def fibonacci(n):
```

```
    a, b = 0, 1
```

```
    count = 0
```

```
    while count < n:
```

```
        yield a
```

```
        a, b = b, a + b
```

```
        count += 1
```

```
for num in fibonacci(10):
```

```
    print(num) # 0, 1, 1, 2, 3, 5, 8, 13, 21, 34
```

Generator maintains state between yields

```
def counter():
```

```
    print("Starting")
```

```
    yield 1
```

```
    print("Between 1 and 2")
```

```
    yield 2
```

```
    print("Between 2 and 3")
```

```
    yield 3
```

```
    print("Ending")
```

```
gen = counter()

print(next(gen)) # Starting, then returns 1

print(next(gen)) # Between 1 and 2, then returns 2

print(next(gen)) # Between 2 and 3, then returns 3
```

print(next(gen)) # Ending, then StopIteration

Practical example: Reading large files

```
def read_large_file(file_path):
```

```
    """Memory-efficient file reading"""

    with open(file_path) as file:

        for line in file:

            yield line.strip()
```

for line in read_large_file('huge_file.txt'):

**process(line) # Only one line in memory
at a time**

Generator pipeline

```
def numbers():
```

```
for i in range(10):  
  
    yield i
```

```
def squares(seq):
```

```
    for num in seq:  
  
        yield num ** 2
```

```
def evens(seq):
```

```
    for num in seq:  
  
        if num % 2 == 0:  
  
            yield num
```

Chain generators

```
pipeline = evens(squares(numbers()))  
  
print(list(pipeline)) # [0, 4, 16, 36, 64]
```

send() method - sending values to generators

```
def echo():
```

```
    while True:  
  
        received = yield  
  
        print(f'Received: {received}')
```

```
gen = echo()

next(gen) # Prime the generator

gen.send("Hello") # Received: Hello

gen.send("World") # Received: World
```

Generator with return value

```
def gen_with_return():
```

```
    yield 1

    yield 2

    return "Done"
```

```
gen = gen_with_return()

print(next(gen)) # 1

print(next(gen)) # 2

try:
```

```
    next(gen)
```

```
except StopIteration as e:
```

```
    print(e.value) # "Done"
```

Generator expression (like list comprehension but lazy)

```
squares_gen = (x ** 2 for x in range(10))

print(next(squares_gen)) # 0
```

```
print(next(squares_gen)) # 1
```

Benefits of Generators:

Memory efficient: Don't store all values in memory

Lazy evaluation: Compute values only when needed

Infinite sequences: Can represent infinite data streams

Pipeline processing: Chain operations efficiently

Stateful iteration: Maintain state between calls

When to use:

Processing large files

Infinite sequences

Data pipelines

When you don't need all values at once

5. Context Managers

Q18: What are context managers and how do you create custom ones?

Answer:

Context managers manage resources (files, locks, connections) ensuring proper acquisition and release using the with statement.

Built-in context manager example

Without context manager (manual cleanup)

```
file = open('test.txt', 'w')
```

try:

```
file.write('Hello')
```

finally:

```
file.close() # Must remember to close
```

With context manager (automatic cleanup)

with open('test.txt', 'w') as file:

```
file.write('Hello')
```

File automatically closed

Custom context manager using enter and exit

class FileManager:

```
def __init__(self, filename, mode):

    self.filename = filename

    self.mode = mode

    self.file = None

def __enter__(self):

    print(f"Opening {self.filename}")

    self.file = open(self.filename, self.mode)

    return self.file
```



```
def __exit__(self, exc_type, exc_val, exc_tb):

    print(f"Closing {self.filename}")

    if self.file:

        self.file.close()

    # Handle exceptions

    if exc_type is not None:

        print(f"Exception: {exc_type}, {exc_val}")

    # Return False to propagate exception, True to suppress

    return False
```

Usage

with FileManager('test.txt', 'w') as f:

```
f.write('Hello World')
```

Opening test.txt

Closing test.txt

Custom context manager using contextlib

```
from contextlib import contextmanager
```

@contextmanager

def file_manager(filename, mode):

```
    print(f"Opening {filename}")

    file = open(filename, mode)

    try:

        yield file # Control returned to 'with' block

    finally:

        print(f"Closing {filename}")

        file.close()
```

with file_manager('test.txt', 'w') as f:

```
    f.write('Hello')
```

Database connection example

@contextmanager

def database_connection(db_name):

```
    print(f"Connecting to {db_name}")

    connection = f"Connection to {db_name}" # Simulated

    try:

        yield connection

    finally:
```

```
print(f"Closing connection to {db_name}")
```

with database_connection('mydb') as conn:

```
print(f"Using {conn}")
```

Connecting to mydb

Using Connection to mydb

Closing connection to mydb

Timer context manager

```
import time
```

```
class Timer:
```

```
    def __enter__(self):
```

```
        self.start = time.time()
```

```
        return self
```

```
    def __exit__(self, exc_type, exc_val, exc_tb):
```

```
        self.end = time.time()
```

```
        self.elapsed = self.end - self.start
```

```
print(f"Elapsed time: {self.elapsed:.4f} seconds")

return False
```

with Timer():

```
time.sleep(1)

print("Doing work...")
```

Doing work...

Elapsed time: 1.0012 seconds

Using contextlib version

@contextmanager

def timer():

```
start = time.time()

yield

end = time.time()

print(f"Elapsed: {end - start:.4f} seconds")
```

with timer():

```
time.sleep(0.5)
```

Multiple context managers

with open('input.txt', 'r') as infile, open('output.txt', 'w') as outfile:

```
for line in infile:

    outfile.write(line.upper())
```

Exception handling in context managers

class ErrorHandler:

```
def __enter__(self):

    print("Entering")

    return self

def __exit__(self, exc_type, exc_val, exc_tb):

    if exc_type is ValueError:

        print("Handled ValueError")

        return True # Suppress the exception

    return False # Propagate other exceptions
```

with ErrorHandler():

```
    print("Inside block")

    raise ValueError("Test error") # Suppressed
```

print("Continues execution")

Entering

Inside block

Handled ValueError

Continues execution

Lock example

```
import threading
```

```
lock = threading.Lock()
```

```
with lock: # lock.acquire() called
```

```
    # Critical section
```

```
    pass
```

lock.release() called automatically

Temporary directory example

```
import tempfile
```

```
import os
```

```
@contextmanager
```

```
def temp_directory():
```

```
temp_dir = tempfile.mkdtemp()

try:

    yield temp_dir

finally:

    # Cleanup

    import shutil

    shutil.rmtree(temp_dir)
```

with temp_directory() as tmpdir:

```
print(f"Working in {tmpdir}")

# Create files in tmpdir
```

tmpdir automatically deleted

When to use context managers:

File operations

Database connections

Network connections

Locks and synchronization primitives

Transaction management

Temporary resources

Setup and teardown operations

Key Points:

enter: Setup, returns resource

exit: Cleanup, receives exception info

Return True in **exit** to suppress exceptions

@contextmanager decorator is simpler for basic cases

6. Error Handling & Exceptions

Q19: Explain exception handling with try/except/else/finally.

Answer:

Basic try-except

try:

```
result = 10 / 0
```

except ZeroDivisionError:

```
print("Cannot divide by zero")
```

Multiple except blocks

try:

```
value = int("not a number")
```

except ValueError:

```
print("Invalid integer")
```

except TypeError:

```
print("Type error")
```

Catching multiple exceptions

try:

```
# Some code
```



```
pass
```

except (ValueError, TypeError) as e:

```
print(f"Error: {e}")
```

Generic exception (catch-all) - use sparingly

try:

```
# Some code
```

```
pass
```

except Exception as e:

```
print(f"Unexpected error: {e}")
```

try-except-else-finally

def divide(a, b):

```
try:
```

```
    result = a / b
```

```
except ZeroDivisionError:
```

```
    print("Cannot divide by zero")
```

```
    return None
```

```
except TypeError:
```

```
print("Invalid types for division")

return None

else:

    # Executed only if no exception occurred

    print("Division successful")

    return result

finally:

    # Always executed, even if return in try/except/else

    print("Cleanup code")
```

```
print(divide(10, 2))
```

Division successful

Cleanup code

5.0

```
print(divide(10, 0))
```

Cannot divide by zero

Cleanup code

None

Real-world example: File handling

```
def read_file(filename):
```

```
    try:

        file = open(filename, 'r')

    except FileNotFoundError:

        print(f"File {filename} not found")

        return None

    except PermissionError:

        print(f"No permission to read {filename}")

        return None

    else:

        # File opened successfully

        content = file.read()

        return content

    finally:

        # Ensure file is closed

        if 'file' in locals():

            file.close()
```

```
print("File closed")
```

Raising exceptions

```
def validate_age(age):
```

```
    if age < 0:

        raise ValueError("Age cannot be negative")

    if age > 150:

        raise ValueError("Age seems invalid")

    return age
```

```
try:
```

```
    validate_age(-5)
```

```
except ValueError as e:
```

```
    print(f"Validation error: {e}")
```

Custom exceptions

```
class InsufficientFundsError(Exception):
```

```
    """Raised when account has insufficient funds"""

    def __init__(self, balance, amount):

        self.balance = balance
```

```
self.amount = amount
```

```
super().__init__(f"Insufficient funds: balance={balance}, attempted={amount}")
```

```
class BankAccount:
```

```
    def __init__(self, balance):
```

```
        self.balance = balance
```

```
    def withdraw(self, amount):
```

```
        if amount > self.balance:
```

```
            raise InsufficientFundsError(self.balance, amount)
```

```
        self.balance -= amount
```

```
        return self.balance
```

```
account = BankAccount(100)
```

```
try:
```

```
    account.withdraw(150)
```

```
except InsufficientFundsError as e:
```

```
    print(f"Error: {e}")
```

```
    print(f"Your balance: ${e.balance}, You tried: ${e.amount}")
```

Exception chaining (raise from)

```
def process_data(data):
```

```
try:

    return int(data)

except ValueError as e:

    raise TypeError("Data must be numeric") from e
```

try:

```
process_data("abc")
```

except TypeError as e:

```
print(f"Error: {e}")

print(f"Caused by: {e.__cause__}")
```

Re-raising exceptions

def risky_operation():

```
try:

    # Some operation

    raise ValueError("Something went wrong")

except ValueError:

    print("Logging error...")

    raise # Re-raise the same exception
```

try:

```
risky_operation()
```

```
except ValueError as e:
```

```
print(f"Caught: {e}")
```

Exception hierarchy

////

BaseException

 SystemExit

 KeyboardInterrupt

 GeneratorExit

 Exception

 StopIteration

 ArithmeticError

 ZeroDivisionError

 OverflowError

 AttributeError

 EOFError

 ImportError

 LookupError

 IndexError

 KeyError

`NameError`

`OSError`

`FileNotFoundError`

`PermissionError`

`TypeError`

`ValueError`

`RuntimeError`

////

Best practices

1. Be specific - catch specific exceptions

2. Don't catch `BaseException` (catches system exits)

3. Use `finally` for cleanup (or context managers)

4. Don't use bare except: (catches everything including KeyboardInterrupt)

Good

try:

```
value = int(input("Enter number: "))
```

except ValueError:

```
print("Invalid number")
```

Bad

try:

```
value = int(input("Enter number: "))
```

except: # Catches KeyboardInterrupt, SystemExit, etc.

```
print("Invalid number")
```

EAFP vs LBYL

EAFP (Easier to Ask for Forgiveness than Permission) - Pythonic

try:

```
value = my_dict['key']
```

except KeyError:

```
value = None
```

LBYL (Look Before You Leap)

if 'key' in my_dict:

```
value = my_dict['key']
```

else:

```
value = None
```

EAFP is preferred in Python (faster when key usually exists)

Key Points:

try: Code that might raise an exception

except: Handle specific exceptions

else: Runs only if no exception occurred

finally: Always runs (cleanup code)

Use specific exceptions, not bare except:

EAFP is more Pythonic than LBYL

7. Scope & Namespaces

Q20: Explain the LEGB rule and scope in Python.

Answer:

LEGB stands for: Local Enclosing Global Built-in

Python searches for variables in this order.

Built-in scope

```
print(len([1, 2, 3])) # 'len' is in built-in scope
```

Global scope

```
global_var = "I'm global"
```

```
def outer_function():
```

```
    # Enclosing scope
```

```
    enclosing_var = "I'm in enclosing scope"
```

```
    def inner_function():
```

```
        # Local scope
```

```
        local_var = "I'm local"
```

```
        # Access in LEGB order
```

```
        print(local_var)    # Local
```

```
        print(enclosing_var) # Enclosing
```

```
        print(global_var)   # Global
```

```
        print(len([1, 2]))  # Built-in
```

```
    inner_function()
```

```
outer_function()
```

Example demonstrating scope lookup

```
x = "global x"
```

```
def test():
```

```
    x = "local x" # Shadows global x
```

```
    print(x) # local x
```

```
test()
```

```
print(x) # global x
```

global keyword - modify global variable

```
count = 0
```

```
def increment():
```

```
    global count # Without this, would create local variable
```

```
    count += 1
```

```
increment()
```

```
print(count) # 1
```

nonlocal keyword - modify enclosing scope variable

```
def outer():
```

```
    x = "outer x"
```

```
def inner():  
  
    nonlocal x # Modify enclosing scope, not create new local  
  
    x = "inner x"  
  
    print("Before:", x) # outer x  
  
    inner()  
  
    print("After:", x) # inner x
```

```
outer()
```

Common mistake: Forgetting global/nonlocal

```
counter = 0
```

```
def increment_wrong():
```

```
    counter += 1 # UnboundLocalError: local variable referenced before assignment
```

**Why? Python sees assignment, assumes
counter is local,**

but tries to read it before assignment

Correct version

```
def increment_correct():
```

```
    global counter
```

```
    counter += 1
```

Namespace example

```
x = 1 # Global namespace
```

```
def func():
```

```
    x = 2 # Local namespace
```

```
    print(f"Local x: {x}")
```

```
func() # Local x: 2
```

```
print(f"Global x: {x}") # Global x: 1
```

Accessing built-in namespace

```
import builtins
```

```
print(dir(builtins)) # See all built-in names
```

Shadowing built-ins (don't do this!)

```
list = [1, 2, 3] # Shadows built-in 'list'
```

list("abc") # TypeError: 'list' object is not callable

Fix by deleting or using different name

```
del list
```

```
result = list("abc") # Now works
```

Class scope doesn't follow LEGB

```
class MyClass:
```

```
    x = 10 # Class variable
```

```
    def method(self):
```

```
        # Cannot access x directly
```

```
        # print(x) # NameError
```

```
        print(self.x) # Must use self.x
```

```
        print(MyClass.x) # Or class name
```

Closure example showing enclosing scope

```
def make_multiplier(n):
```

```
    def multiplier(x):
```

```
        # 'n' is in enclosing scope, accessible here
```

```
        return x * n
```

```
return multiplier
```

```
times_3 = make_multiplier(3)
```

```
times_5 = make_multiplier(5)
```

```
print(times_3(10)) # 30
```

```
print(times_5(10)) # 50
```

Each closure has its own enclosing scope

Practical example: Counter with enclosing scope

```
def make_counter():
```

```
    count = 0 # Enclosing scope
```

```
    def increment():
```

```
        nonlocal count
```

```
        count += 1
```

```
        return count
```

```
    def decrement():
```

```
        nonlocal count
```

```
        count -= 1
```



```
    return count

def get_count():

    return count

return increment, decrement, get_count
```

```
inc, dec, get = make_counter()
```

```
print(inc()) # 1
```

```
print(inc()) # 2
```

```
print(dec()) # 1
```

```
print(get()) # 1
```

locals() and globals()

```
global_var = "global"
```

```
def show_namespaces():
```

```
    local_var = "local"

    print("Locals:", locals()) # {'local_var': 'local'}

    print("Globals:", 'global_var' in globals()) # True
```

```
show_namespaces()
```

Summary:

Local (L): Variables defined in the current function

Enclosing (E): Variables in the enclosing function (closures)

Global (G): Variables at module level

Built-in (B): Python's built-in names

Key Rules:

Variables are local by default when assigned in a function

Use global to modify global variables

Use nonlocal to modify enclosing scope variables

Reading doesn't need global/nonlocal, only writing does

Class scope is separate from LEGB

This covers the Core Python concepts with interview questions and detailed answers.

Would you like me to continue with more advanced topics or create a separate file for other sections?